AMRITA VISHWA VIDYAPEETHAM
DEEMED TO BE UNIVERSITY

Amrita School of AI
COIMBATORE CAMPUS

HIGH-PERFORMANCE AND CLOUD COMPUTING

Unit 1: Parallel Architecture and Performance

How parallel machines are organised, how to design an algorithm for them, and how to say in advance how much faster it can possibly get.

Prepared by **Dr. Abhijith Anandakrishnan**
Assistant Professor, Amrita School of AI

COURSE
23AID304 High-Performance and Cloud Computing

UNIT
1 of 4

SYLLABUS
Architecture and OS concepts, parallel architecture, shared and distributed memory, parallel algorithms, performance metrics

PREREQUISITE
Unit 0

OFFERING
Odd Semester 2026-27

What this unit is for

Unit 0 argued that parallelism is compulsory. This unit gives you the vocabulary to describe parallel machines, a method for designing parallel algorithms, and, most importantly, the arithmetic to predict performance *before* you write code.

That last point is what separates high-performance computing from ordinary programming. Anyone can write a loop and time it. The skill worth having is being able to say "this kernel moves 8 bytes per flop, the machine delivers 200 GB/s, so 25 GFLOP/s is the ceiling and I am already at 22, therefore stop optimising" and be right.

BY THE END OF THIS UNIT YOU WILL BE ABLE TO

- Classify a machine using Flynn's taxonomy and say what it implies for programming. - Explain the difference between shared and distributed memory, and choose the right model for a problem. - Apply Foster's four-step method to decompose a problem into parallel tasks. - Compute speedup, efficiency, and the serial fraction from measurements. - Use Amdahl's law and Gustafson's law correctly, and say when each applies. - Distinguish strong from weak scaling and design a scaling study. - Compute arithmetic intensity and use the roofline model to decide whether a kernel is memory bound or compute bound, and whether it is worth optimising.

KEY TERMS IN THIS UNIT

Flynn's taxonomy · SISD · SIMD · MISD · MIMD · SPMD · shared memory · distributed memory · UMA · NUMA · cache coherence · interconnect · bisection bandwidth · diameter · latency · Foster's method · partitioning · granularity · load balance · speedup · efficiency · serial fraction · Amdahl's law · Gustafson's law · strong scaling · weak scaling · arithmetic intensity · roofline · ridge point

Classifying parallel machines

Flynn's taxonomy

Michael Flynn classified computers in 1966 by how many **instruction streams** and how many **data streams** they have. The scheme is sixty years old and still the standard vocabulary.

	SINGLE DATA STREAM	MULTIPLE DATA STREAMS
Single instruction stream	**SISD** classical sequential processor	**SIMD** one instruction, many data items
Multiple instruction streams	**MISD** rare; fault-tolerant pipelines	**MIMD** independent processors, the general case

- **SISD** is the textbook von Neumann machine of Unit 0. No parallelism visible to the programmer, though the hardware still pipelines and superscalar-issues underneath.
- **SIMD** is one instruction applied to a whole vector. Your processor's AVX units are SIMD. So, in a modified form, is every GPU.
- **MISD** has essentially no commercial examples. It is included for completeness and appears in exams.
- **MIMD** is every multicore processor and every cluster. Each core runs its own instruction stream on its own data.

DEFINITION · SPMD, THE MODEL YOU WILL ACTUALLY WRITE

Single Program, Multiple Data is a *programming style* on MIMD hardware: every processor runs the same program, but branches on its own identity to work on a different part of the data.

```
c int rank; MPI_Comm_rank(MPI_COMM_WORLD, &rank); if (rank == 0) collect_results(); else compute_my_chunk(rank);
```

Every MPI program in Unit 2 is SPMD, and so is every CUDA kernel in Unit 3. SPMD is not a Flynn category, and confusing it with SIMD is a common exam mistake: SIMD is a hardware property, SPMD is how you write the program.

BEYOND THE SYLLABUS · SIMT, THE GPU VARIANT

NVIDIA calls its model **SIMT**, single instruction multiple thread. Threads are grouped into **warps** of 32 that share one instruction pointer, so the hardware is SIMD, but each thread has its own registers and can branch independently. When threads in a warp take different branches the hardware executes both paths and masks off the inactive threads, which is called **warp divergence** and costs real performance. Unit 3 covers it in detail.

Shared memory versus distributed memory

This is the distinction that determines which library you use, and it follows directly from processes and threads in Unit 0.

	SHARED MEMORY	DISTRIBUTED MEMORY
Address space	One, visible to all cores	One per node, private
Communication	Write a variable, another core reads it	Explicit send and receive
Programmed with	OpenMP, pthreads	MPI
Scales to	One node: tens to hundreds of cores	Thousands of nodes
Limited by	Memory bandwidth, coherence traffic	Network latency and bandwidth
Hardest bug	Data race	Deadlock, mismatched messages
On `asaicompute`	Within one node, up to 96 cores	Across the three nodes

Shared memory: UMA and NUMA

In a **uniform memory access (UMA)** machine every core reaches every memory location at the same cost. Small multicore chips approximate this.

In a **non-uniform memory access (NUMA)** machine memory is attached to sockets or to groups of cores, and a core reaches its own memory faster. Every large server is NUMA, including the AMD EPYC processors in `asaicompute`.

PITFALL · THE FIRST-TOUCH RULE

On Linux, a page of memory is physically placed near the core that **first writes** to it, not the core that called `malloc`. So this is slow:

```c
double *a = malloc(n * sizeof *a);
for (i = 0; i < n; i++) a[i] = 0.0;   /* thread 0 touches everything */

#pragma omp parallel for
for (i = 0; i < n; i++) a[i] = f(i);  /* all memory is on one socket */
```

Every page ends up near whichever core ran the serial initialisation, so most threads read across the interconnect. The fix is to initialise with the same parallel loop pattern used later, so each thread first-touches the pages it will own. On a two-socket machine this alone can be worth 30 to 40 percent.

Cache coherence

If two cores cache the same memory location and one writes to it, the other must not keep reading the stale copy. **Cache coherence** protocols, usually a variant of MESI (Modified, Exclusive, Shared, Invalid), maintain this automatically.

Coherence is a service you get for free and pay for in traffic. Every write to a line that other cores hold causes invalidation messages. This is the mechanism behind **false sharing**, the subject of one of the nastiest bugs in Unit 2:

PITFALL · FALSE SHARING, PREVIEWED

```c
double partial[NTHREADS]; /* 8 doubles = ONE cache line */

#pragma omp parallel
{
    partial[omp_get_thread_num()] += x; /* looks perfectly independent */
}
```

Each thread writes its own element, so there is no data race and the answer is correct. But all eight elements share one 64-byte cache line, so every write by any thread invalidates the line in all the others. The line ping-pongs between cores and the loop can run **slower than serial**. The fix is padding, and we do it properly in Unit 2.

Interconnects

Distributed-memory machines are defined by their network. Three properties matter:

DEFINITION · LATENCY, BANDWIDTH, AND TOPOLOGY METRICS

Latency is the time for a zero-length message to arrive, typically 1 to 5 microseconds on InfiniBand, 20 to 50 on plain Ethernet. **Bandwidth** is the sustained rate for large messages. **Diameter** is the largest number of hops between any two nodes. **Bisection bandwidth** is the bandwidth across the worst-case cut that splits the machine in half, and it bounds any communication pattern where everyone talks to everyone.

TOPOLOGY	DIAMETER	BISECTION BANDWIDTH	NOTES
Bus	1	one link	Does not scale past a few nodes
Ring	$N/2$	2 links	Cheap; the basis of NCCL's ring all-reduce in Unit 4
2-D mesh	$2(\sqrt{N} - 1)$	$\sqrt{N}$ links	Natural fit for stencil problems
Torus	$\sqrt{N}$	$2\sqrt{N}$ links	Mesh with wraparound; used by several top machines
Hypercube	$\log_2 N$	$N/2$ links	Excellent properties, expensive wiring
Fat tree	$2 \log_k N$	full	The standard for modern clusters

A useful model for the cost of sending n bytes:

$$T(n) = \alpha + n / \beta$$

α latency (startup cost, per message) · β bandwidth (bytes per second)

This tiny formula explains most MPI performance advice. Because α is paid per *message* regardless of size, sending one 1 MB message is far cheaper than sending a thousand 1 KB messages. **Aggregate your communication.**

WORKED EXAMPLE · WHEN DOES MESSAGE SIZE STOP MATTERING?

Take α = 2 µs and β = 10 GB/s. The transfer time equals the latency when $n/\beta = \alpha$, that is $n = 2 \times 10^{-6} \times 10 \times 10^{9}$ = **20 000 bytes**.

Below about 20 KB you are paying mostly for latency, and doubling the message size costs almost nothing. Above it you are paying for bandwidth. This crossover is why MPI implementations switch protocols around exactly this size, and why batching small messages is the first thing to try when MPI code is slow.

Designing a parallel algorithm

Parallel programming is not "add `#pragma omp parallel for` and hope". Ian Foster's **PCAM** method gives a repeatable four-step design process.

Step 1: Partitioning

Split the computation into the smallest sensible pieces, ignoring how many processors you have. Two ways in:

- **Domain decomposition**: split the *data* first, then assign the computation that touches each piece. Split an image into tiles, a grid into blocks, a matrix into row bands. This is the usual choice in scientific computing.
- **Functional decomposition**: split the *computation* first. A climate model might run atmosphere, ocean, and ice as separate concurrent components.

Step 2: Communication

Work out what data must move between tasks.

- **Local** communication: each task talks to a few neighbours, as in a stencil.
- **Global** communication: every task contributes to one result, as in a sum.
- **Structured** versus **unstructured**: a regular grid versus an irregular mesh.

Step 3: Agglomeration

Combine small tasks into larger ones to reduce communication and overhead. This is where **granularity** is decided.

> **DEFINITION · GRANULARITY**
>
> The ratio of computation to communication in a task.
>
> **Fine-grained**: little work per task, frequent communication. Flexible, good load balance, high overhead.
> **Coarse-grained**: much work per task, rare communication. Low overhead, risk of imbalance.
>
> Getting this wrong in either direction is the most common cause of a parallel program that is slower than the serial one.

Step 4: Mapping

Assign the agglomerated tasks to physical processors, aiming to balance load and minimise communication. On a cluster this is partly the scheduler's job, but you choose the decomposition that makes a good mapping possible.

WORKED EXAMPLE · PCAM APPLIED: HEAT DIFFUSION ON A 2-D GRID

Problem. Update every cell of an $N \times N$ grid from its four neighbours, repeatedly.

Partition. One task per cell. N^2 tasks.

Communicate. Each cell needs its four neighbours' values from the previous step. Local, structured communication.

Agglomerate. One cell per task is far too fine: the communication would dwarf one floating-point update. Group cells into blocks. A $b \times b$ block does b^2 work and communicates its $4b$ boundary values, so the compute-to-communicate ratio is $b/4$ and grows with block size. Bigger blocks are more efficient but give less flexibility to balance load.

Map. Assign one block per core, arranged so that neighbouring blocks sit on neighbouring cores. Each core exchanges **halo** or **ghost** cells with its neighbours once per step.

This decomposition is exactly what you implement in the MPI stencil exercise, and the same reasoning produces the tiled matrix multiply in Unit 3.

Load balance

$$T_{\text{parallel}} = \max_i T_i$$

A parallel program finishes when its **slowest** task finishes. If one core has twice the work, the other 95 cores wait, and you have bought a 96-core machine to run at the speed of two.

STRATEGY	HOW IT WORKS	BEST FOR
Static	Divide work up front, equal chunks	Uniform work per item
Dynamic	Workers take the next chunk when free	Uneven or unpredictable work
Guided	Chunks start large and shrink	Mildly uneven work; less overhead than dynamic
Work stealing	Idle workers take work from busy ones' queues	Irregular, recursive problems

OpenMP exposes these directly as `schedule(static)`, `schedule(dynamic)`, and `schedule(guided)`, which you will use in Unit 2.

Performance metrics

Now the arithmetic. These definitions are examinable and, more usefully, they are how you will report every result in this course.

Speedup and efficiency

DEFINITION · SPEEDUP

$$S(N) = T_1 / T_N$$

T_1 time on one processor · T_N time on N processors

T_1 must be the time of **the best serial algorithm**, not your parallel program run with one thread. Using the latter flatters your result by including parallel overhead in the baseline.

DEFINITION · EFFICIENCY

$$E(N) = S(N) / N$$

The fraction of the machine you are actually using. $E = 1$ is perfect. Below about 0.5 you are wasting more than half the hardware, and it is worth asking whether fewer cores would be a better use of the allocation.

COMMON MISCONCEPTION · SUPERLINEAR SPEEDUP IS IMPOSSIBLE

It is not. $S(N) > N$ happens legitimately when the problem's data is split across N processors and each piece now fits in cache, where it did not before. You gain a memory-hierarchy effect on top of the parallelism. It is real, but it is also a common symptom of a badly chosen serial baseline, so check that first.

Amdahl's law

Suppose a fraction s of the runtime is inherently serial and cannot be parallelised. Then however many processors you add:

$$S(N) = 1 / (s + (1 - s) / N) \quad \longrightarrow \quad S_\infty = 1 / s$$

s serial fraction · $(1 - s)$ parallel fraction

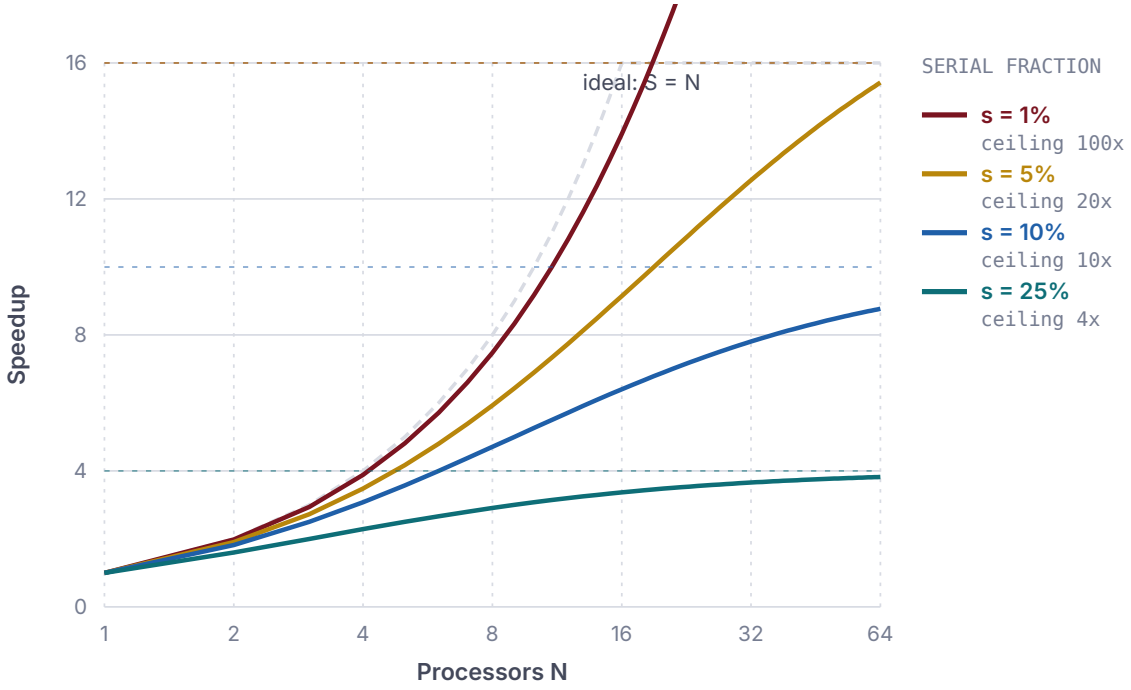

Figure 1.1 Amdahl's law. Each curve is a different serial fraction, and each dashed horizontal line is that curve's ceiling. With five percent serial code the speedup can never exceed twenty, no matter how large the machine. The gap between the curves and the dashed diagonal is the machine you paid for and did not use.

The numbers are brutal and worth memorising:

SERIAL FRACTION S	MAXIMUM SPEEDUP	EFFICIENCY AT N = 64
0%	unbounded	100%
1%	100	61%
5%	20	24%
10%	10	14%
25%	4	6%
50%	2	3%

WORKED EXAMPLE · READING AMDAHL BACKWARDS

You measure T_1 = 100 s and T_8 = 20 s, so $S(8) = 5$ and $E = 0.63$. What is the serial fraction?

Rearranging Amdahl's law gives the **Karp-Flatt metric**:

$$e = (1/S - 1/N) / (1 - 1/N)$$

Here e = (1/5 − 1/8) / (1 − 1/8) = (0.200 − 0.125) / 0.875 = **0.086**.

So about 8.6 percent of the work is serial, and the ceiling is 1/0.086 ≈ 11.6. Going from 8 to 64 cores would take you from 5 to at most about 9. That is the calculation that tells you to fix the serial part instead of asking for more cores, and it is exactly what a scaling study is for.

Gustafson's law

Amdahl assumes the problem size is fixed. In practice nobody buys a bigger machine to solve the same problem faster; they buy it to solve a *bigger* problem. If the parallel part of the work grows with the machine while the serial part stays constant, the picture changes completely:

$$S(N) = s + N \cdot (1 - s) = N - s \cdot (N - 1)$$

This is **linear in *N***, with no ceiling. With s = 0.05 and N = 1024, Amdahl says 20 and Gustafson says 973.

COMMON MISCONCEPTION · AMDAHL AND GUSTAFSON CONTRADICT EACH OTHER

They do not. They answer different questions.

Amdahl: *fixed problem*, more processors. How much sooner does it finish? Answer: there is a hard ceiling.

Gustafson: *fixed time budget*, more processors. How much bigger a problem can I solve? Answer: proportionally bigger.

Amdahl governs strong scaling. Gustafson governs weak scaling. Both are correct; the question is which one matches what you are actually doing.

Strong and weak scaling

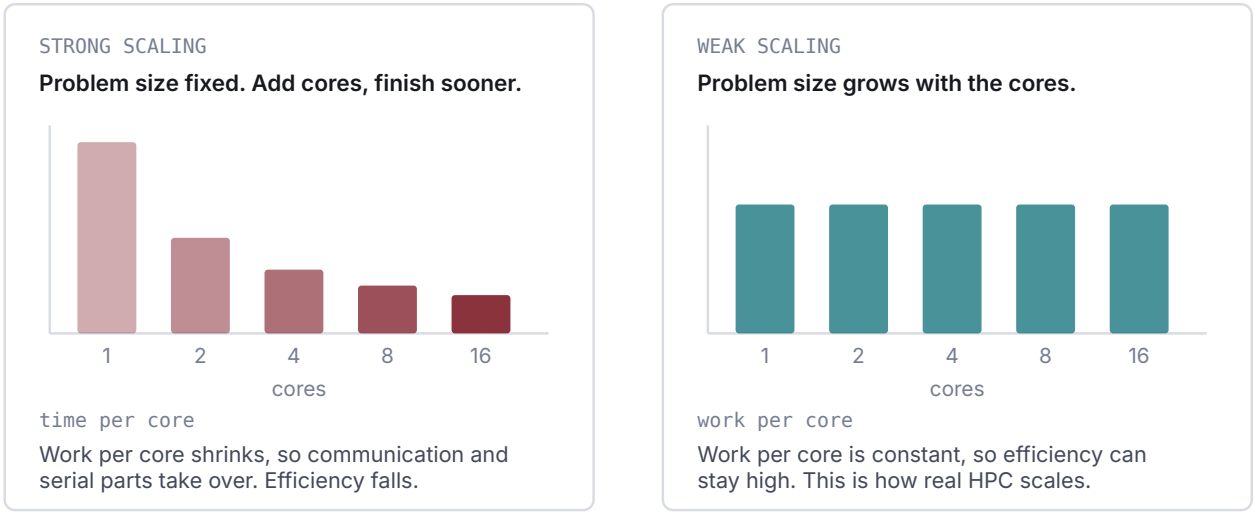

Figure 1.2 The two kinds of scaling study. In a strong-scaling study the total problem is fixed, so each core's share of the work shrinks as cores are added and overheads eventually dominate. In a weak-scaling study the problem grows with the core count, so the work per core is constant and efficiency can be maintained.

DEFINITION · STRONG AND WEAK SCALING

Strong scaling: fix the total problem size, increase N, measure how time falls. Governed by Amdahl. Reported as speedup and efficiency.

Weak scaling: fix the problem size *per processor*, increase N so the total problem grows proportionally, measure how time changes. Perfect weak scaling means constant runtime. Governed by Gustafson.

WORKED EXAMPLE · DESIGNING A SCALING STUDY YOU CAN DEFEND

For a strong-scaling study of a 4096 × 4096 matrix multiply:

1. Time the best **serial** implementation. This is T_1.
2. Run at N = 1, 2, 4, 8, 16, 32, 64 threads.
3. At each N: one warm-up, then five timed runs, keep the minimum.
4. Pin threads (`OMP_PROC_BIND=close`) so the operating system does not migrate them mid-run and add noise.
5. Compute $S(N)$ and $E(N)$, and e from Karp-Flatt at each point.
6. Plot S against N with the ideal line, and E against N.
7. State the machine, compiler, flags, problem size, repetitions, and statistic.

An e that is roughly constant across N means you really do have that much serial work. An e that **grows** with N means the loss is not a fixed serial section at all: it is overhead that scales, usually communication or synchronisation. That distinction tells you what to fix, and you cannot see it from the speedup plot alone.

The roofline model

Speedup tells you how well you are using the *cores*. It says nothing about whether you are using the *machine*. A code can scale perfectly to 64 cores and still run at one percent of the hardware's capability.

The **roofline model** fixes that. It is the single most useful performance tool in this course, it is not in the printed syllabus, and you should learn it anyway.

BEYOND THE SYLLABUS · WHY THIS IS HERE

The syllabus lists "performance metrics of parallel algorithms", which traditionally means speedup and efficiency. Those are relative measures: they compare your code against your own serial version. Roofline is an *absolute* measure: it compares your code against what the hardware can do. Without it, "make it faster" has no stopping criterion.

Arithmetic intensity

DEFINITION · ARITHMETIC INTENSITY

$$I = W / Q$$

W floating-point operations performed · Q bytes moved between DRAM and the processor

Measured in FLOP per byte. It is a property of the **algorithm and its implementation**, not of the machine.

Compute it for common kernels:

KERNEL	FLOPS	BYTES FROM DRAM	I (FLOP/BYTE)
Vector copy `y[i]=x[i]`	0	16	0
Vector scale `y[i]=a*x[i]`	1	16	0.06
Vector add `y[i]=x[i]+z[i]`	1	24	0.04
DAXPY `y[i]+=a*x[i]`	2	24	0.08
Dot product `s+=x[i]*y[i]`	2	16	0.125
5-point stencil	5	8 (with good reuse)	0.6
Naive matrix multiply	$2n^3$	$\sim 8n^3$	0.25
Blocked matrix multiply	$2n^3$	$\sim 8n^3/b$	$\sim b/4$
Dense GEMM, well tuned	$2n^3$	$\sim 24n^2$	grows with n

The two matrix multiply rows are the whole point. **Same algorithm, same flops, same answer.** Blocking changes only *which order* the operations happen in, and that changes Q by a factor of b, moving the kernel from hopeless to excellent.

The model

Attainable performance is bounded by two ceilings at once: the machine's peak arithmetic rate, and what its memory system can feed you.

$$\text{Attainable GFLOP/s} = \min(\, P_{\text{peak}} \,,\, \beta \times I \,)$$

P_{peak} peak compute rate · β peak memory bandwidth · I arithmetic intensity

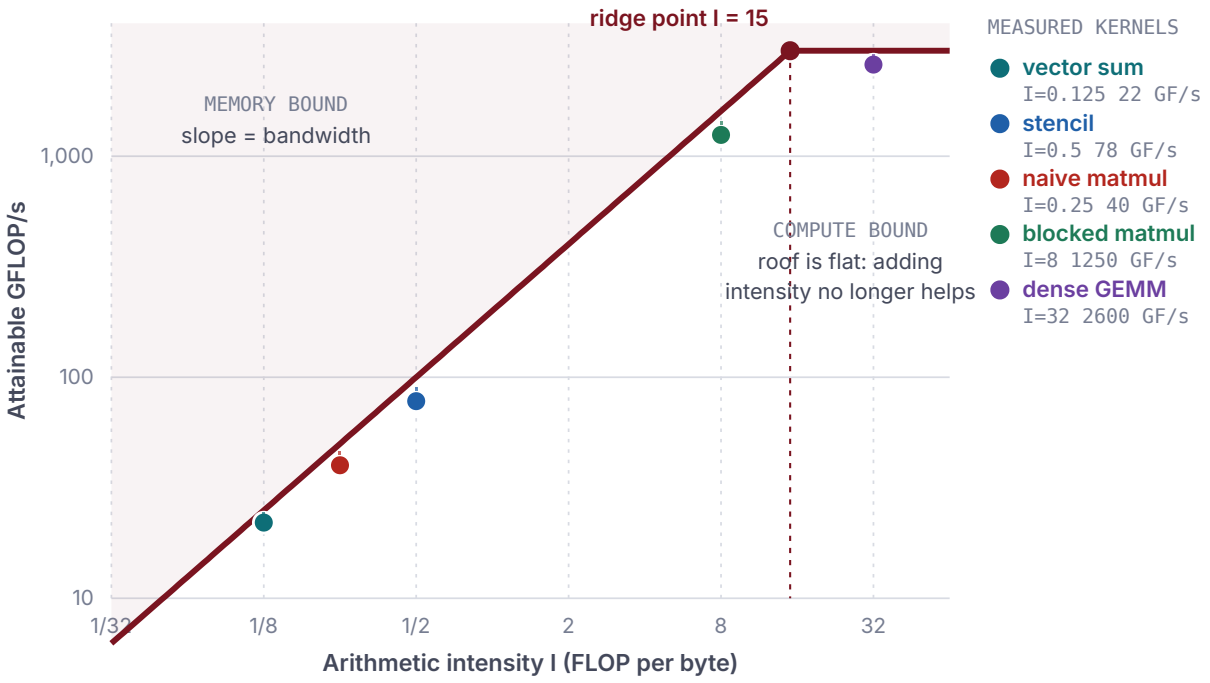

Figure 1.3 The roofline. The sloped section is the bandwidth ceiling, the flat section the compute ceiling, and their meeting point is the ridge. A kernel is plotted at its arithmetic intensity; the dotted line above each point is the headroom that optimisation could recover. Points to the left of the ridge can never reach peak compute however well they are written.

DEFINITION · RIDGE POINT

The arithmetic intensity at which the two ceilings meet, $I_{\text{ridge}} = P_{\text{peak}} / \beta$. Kernels to the **left** are **memory bound**; kernels to the **right** are **compute bound**.

The ridge point is a property of the machine and it has been moving right for thirty years, because compute has improved faster than bandwidth. That drift is the memory wall of Unit 0, drawn as a single number.

Using it to make decisions

This is the part that matters. Given a kernel:

1. Count the flops W, by hand, from the loop body.
2. Count the bytes Q that must come from DRAM, assuming reasonable cache reuse.
3. Compute $I = W/Q$ and find your position on the roofline.
4. Measure what you actually achieve.
5. Compare.

WHERE YOU ARE	WHAT IT MEANS	WHAT TO DO
At the sloped roof	Memory bound, running at the bandwidth limit	Raise I: block, tile, fuse loops, use lower precision. Do **not** optimise arithmetic
Well below the sloped roof	Memory bound and wasting bandwidth	Fix access patterns: stride, alignment, prefetching, NUMA first touch
At the flat roof	Compute bound at peak	You are done. Stop
Well below the flat roof	Compute bound but not vectorising or not using all cores	Check vectorisation reports, thread count, instruction mix

WORKED EXAMPLE · SHOULD I OPTIMISE THIS KERNEL?

A dot product on one `asaicompute` node. Measure or look up: $\beta \approx 200$ GB/s, $P_{\text{peak}} \approx 3000$ GFLOP/s (double precision).

Ridge point: 3000 / 200 = **15 FLOP/byte**.

Dot product: two flops per iteration, two doubles loaded, so $I = 2/16$ = **0.125 FLOP/byte**. That is far to the left of the ridge, so the kernel is firmly memory bound.

Ceiling: 200 GB/s × 0.125 FLOP/byte = **25 GFLOP/s**, which is 0.8 percent of peak compute.

You measure 22 GFLOP/s. That is 88 percent of the achievable ceiling.

Conclusion: stop. No amount of loop unrolling, intrinsics, or clever arithmetic will help, because arithmetic is not the constraint. The only way to go faster is to change the algorithm so it moves fewer bytes, for instance by fusing this operation with a neighbouring one so the data is loaded once and used twice.

Without the roofline you would have spent a week hand-tuning a loop that was already at 88 percent of its ceiling.

PITFALL · PERCENT OF PEAK IS A MISLEADING NUMBER ON ITS OWN

A vendor benchmark quoting "3 percent of peak" sounds terrible and may be perfect. For a kernel with $I = 0.125$ on a machine whose ridge is at 15, the *maximum possible* is 0.8 percent of peak. Always compare against the roofline, never against unqualified peak.

BEYOND THE SYLLABUS · REFINEMENTS WORTH KNOWING

Real roofline plots add more ceilings: a lower compute roof for code that does not vectorise, another for code that does not use fused multiply-add, and extra sloped roofs for L2 and L3 bandwidth rather than DRAM. Each additional roof turns “you are below the line” into a specific diagnosis. Tools such as Intel Advisor and NVIDIA Nsight Compute generate these automatically, and Unit 3 uses the GPU version.

Putting it together

A complete performance analysis of any kernel in this course answers four questions, in this order:

1. **What is the ceiling?** Compute I, place it on the roofline, get the attainable rate. This tells you whether the problem is memory or compute bound.
2. **How close am I on one core?** If far below, fix the serial code first: access pattern, vectorisation, algorithm. Parallelising bad serial code just distributes the badness.
3. **How well does it scale?** Run a scaling study, compute $E(N)$ and the Karp-Flatt e. A rising e means overhead, not serial work.
4. **Is more optimisation worth it?** Compare achieved against attainable. If you are at 90 percent of the roofline, the remaining 10 percent is not where the time should go.

Almost every performance discussion in the rest of this course is one of these four questions.

Self-assessment

1. Classify each of these in Flynn's taxonomy: a single-core 8086; an AVX-512 vector unit; a 96-core EPYC node; a cluster of such nodes. 2. Explain the difference between SIMD and SPMD, and give one example of each. 3. A program allocates an array in a serial loop and then processes it with an OpenMP parallel loop on a two-socket machine. It runs at 60 percent of expected speed. Give the likely cause and the fix. 4. Two threads write to `partial[0]` and `partial[1]` of a `double` array. There is no data race. Why might this still be slow, and what is the fix called? 5. A network has latency 3 µs and bandwidth 12.5 GB/s. At what message size does transfer time equal latency? What does that imply for a program that sends many small messages? 6. Apply PCAM to multiplying two *n* × *n* matrices. State the partition, the communication, a sensible agglomeration, and the mapping. 7. T_1 = 240 s, T_{16} = 22 s. Compute *S*, *E*, and the Karp-Flatt *e*. What is the maximum speedup this code can ever reach? 8. A student reports *S*(4) = 4.6. Give one legitimate explanation and one likely mistake. 9. State Amdahl's law and Gustafson's law, and explain in two sentences why they do not contradict each other. 10. A machine has P_{peak} = 2000 GFLOP/s and *β* = 250 GB/s. Find the ridge point. A kernel performs 3 flops per 24 bytes moved. Is it memory or compute bound, and what is its ceiling? 11. For the kernel in question 10, you measure 28 GFLOP/s. What fraction of the attainable rate is that, and what should you do next? 12. Explain why blocking a matrix multiply changes its arithmetic intensity when it does not change the number of floating-point operations. 13. Your scaling study shows *e* rising from 0.02 at *N* = 4 to 0.11 at *N* = 64. What does this tell you, and what would you investigate? 14. Why is it wrong to use "the parallel program run with one thread" as T_1?

---

1. 8086: SISD. AVX-512 unit: SIMD. 96-core node: MIMD (shared memory). Cluster: MIMD (distributed memory). **2.** SIMD is a hardware property: one instruction operates on several data elements simultaneously, as in an AVX vector add. SPMD is a programming style on MIMD hardware: every processor runs the same program but branches on its rank to handle different data, as in every MPI program. **3.** First touch. The serial initialisation loop caused every page to be placed in the memory attached to one socket, so most threads later read across the interconnect. Fix: initialise the array with a parallel loop using the same schedule and decomposition as the compute loop, so each thread first-touches the pages it will use. **4.** False sharing. `partial[0]` and `partial[1]` are 8 bytes apart and share a 64-byte cache line, so each write invalidates the other core's copy and the line ping-pongs between caches. The fix is padding: give each thread its own cache line, or use a private accumulator combined at the end. **5.** *n* = *α* × *β* = 3×10^{-6} s × 12.5×10^{9} B/s = **37 500 bytes**, about 37 KB. Below that, cost is dominated by per-message latency, so a program sending many small messages pays almost the full latency for each and should aggregate them into fewer, larger messages. **6.** Partition: one task per output element *C*[i][j], each a dot product of a row and a column. Communication: task (i,j) needs row *i* of *A* and column *j* of *B*, so this is global and heavy. Agglomeration: group output into *b* × *b* tiles, so a tile needs *b* rows and *b* columns and does b^2 dot products, raising the compute-to-communicate ratio by *b*. Mapping: a two-dimensional grid of processes, one tile each, so that row and column broadcasts stay within process rows and columns. **7.** *S* = 240/22 = **10.9**. *E* = 10.9/16 = **0.68**. *e* = (1/10.9 − 1/16) / (1 − 1/16) = (0.0917 − 0.0625)/0.9375 = **0.0312**. Maximum speedup is 1/*e* ≈ **32**. **8.** Legitimate: superlinear speedup from cache effects, because each thread's share of the data now fits in its private cache when the whole array did not. Likely mistake: T_1 was measured with the parallel program at one thread, or with an unoptimised build, making the baseline artificially slow. **9.** Amdahl: *S* = 1/(*s* + (1−*s*)/*N*), bounded by 1/*s*, for a **fixed problem size**.

Gustafson: *S* = *N* − *s*(*N*−1), unbounded, for a **fixed runtime** where the problem grows with the machine. They answer different questions, so both are correct; Amdahl governs strong scaling and Gustafson governs weak scaling. **10.** Ridge = 2000/250 = **8 FLOP/byte**. Kernel *I* = 3/24 = **0.125 FLOP/byte**, which is well left of the ridge, so it is **memory bound**. Ceiling = 250 × 0.125 = **31.25 GFLOP/s**. **11.** 28 / 31.25 = **90 percent** of attainable. Stop optimising the arithmetic. The only remaining gains come from raising *I*, for example by fusing this loop with another that reads the same data, or by using single precision to halve the bytes. **12.** The flop count is fixed at 2*n*³, but *Q*, the bytes that must travel from DRAM, is not. Naive multiplication re-reads a row or column from memory for every output element. Blocking loads a *b* × *b* tile once into cache and performs *b* times as much arithmetic on it before evicting it, so *Q* falls by roughly a factor of *b* while *W* is unchanged, and *I* = *W*/*Q* rises proportionally. **13.** A constant *e* would indicate a genuinely serial section. A rising *e* indicates overhead that grows with processor count, which is not what Amdahl's *s* models. Investigate communication volume, synchronisation and barrier costs, load imbalance appearing at higher *N*, and contention for shared bandwidth. **14.** It includes the parallel runtime's overhead in the baseline: thread creation, synchronisation, and any restructuring done to enable parallelism. That inflates *T_1* and therefore inflates the reported speedup. The honest baseline is the best serial algorithm, compiled with the same optimisation, with no parallel scaffolding.

Where to go next

Unit 2 puts all of this into practice. OpenMP realises the shared-memory model, MPI realises the distributed-memory model, and you will apply PCAM to a real decomposition. The roofline reappears as the reason blocking a matrix multiply matters, and false sharing reappears as a bug you will have to find and fix.